
Option Pricing Models

Davide Onorio (64658)
Arnaud Van Hees (64691)
Nicolas Ulysse Arboleda Chavez (64690)
Yuan Chen Ding (64667)

Programming Techniques
Lisbon School of Economics and Management
December 2025

1 Introduction

1.1 Problem Statement

Option pricing is a central problem in the world of finance and, more specifically, in the world of quantitative finance. Derivative securities, such as European and American options, are widely traded in financial markets, and their valuation plays a critical role in trading, risk management, and portfolio construction. Among the most influential models for option pricing are the Black–Scholes–Merton model [2], which provides a closed-form analytical solution. On the other hand, we have the binomial tree model, which offers a discrete-time numerical approach. Despite their widespread use, both models rely on simplifying assumptions, such as constant volatility, no frictions in the markets, and lognormally distributed asset prices, that may not hold in real markets. As a result, model prices often deviate from observed market option prices, raising important questions about model accuracy and practical applicability. The objective of this project is to implement the two models in Python and let the user have some freedom in which stock's options and type of options are being priced.

1.2 Motivation

There are two main motivations for this study:

First, from a theoretical perspective, the Black–Scholes–Merton and binomial models form the foundation of modern derivative pricing theory. We have seen these two models in the course Mathematical Methods for finance, and to deepen our understanding of their assumptions, mathematical structure, and relationship to each other is essential for our learning and any future jobs prospects. Implementing these models from scratch reinforces our learning in other courses, mathematics, and programming techniques.

Second, option pricing provides an ideal framework for developing practical quantitative skills. Implementing pricing algorithms in Python requires translating mathematical formulas into efficient and reliable code, handling numerical stability issues, and structuring programs for clarity and extensibility. These skills can be directly transferred to more advanced numerical methods used in practice, such as Monte Carlo simulation methods.

Overall, this project serves as a bridge between our theoretical knowledge and practical use of real life data and its applications.

1.3 Literature Review

The valuation of financial derivatives has been a central topic in financial economics for several decades. The modern theory of option pricing began with the pioneering work of Black, Scholes and Merton (1973), who introduced a continuous-time framework for pricing European options [2]. The Black–Scholes model derives a closed-form solution by assuming that the price of the underlying asset follows geometric Brownian motion with constant volatility, and that markets are frictionless and free of arbitrage. Despite these simplifying assumptions, the model has had a profound impact on both academic research and market practice, and remains a benchmark for option valuation.

The Cox, Ross and Rubinstein (1979) [3] binomial model later provided a discrete-time framework for developing the binomial option pricing model, representing the evolution of the underlying asset price through a recombining tree in discrete time. The binomial framework provides an intuitive and flexible approach to option pricing, allowing for numerical valuation through backward induction. Importantly, Cox et al. demonstrated that,

with appropriate parameterisation, the binomial model converges to the Black–Scholes price as the number of time steps increases. This result established a strong theoretical link between discrete-time and continuous-time pricing models [3].

However, empirical studies have consistently documented deviations between Black–Scholes prices and observed market option prices. One of the most well-known findings is the volatility smile or skew, which was first documented in equity index options after the 1987 market crash [5]. This phenomenon reflects the fact that implied volatility varies systematically across strike prices and maturities, contradicting the Black–Scholes assumption of constant volatility. These empirical observations have motivated a large body of research on alternative models, including stochastic volatility models, jump-diffusion models [4], and local volatility frameworks.

1.4 Objectives

1. How does the binomial option pricing model perform relative to the Black–Scholes model and market prices?
2. What is the convergence behavior of the binomial model toward the Black–Scholes price as the number of time steps increases?

2 Methodology

2.1 Data Sources and Description

The data used in this project are obtained from the Python library `yfinance`, which provides a convenient interface to historical financial market data sourced from Yahoo Finance. This data source is widely used in academic and applied finance projects for educational and exploratory purposes, as it offers access to equity prices and interest rate proxies all in the same place.

2.1.1 Asset Price

Historical prices of the underlying assets are retrieved using `yfinance` for the selected equity tickers. The primary input for option models is the spot price of the underlying asset, which is proxied by the most recent adjusted closing price available at the time of data extraction. It is important to note that, after being imported, the historical stock price series is expressed as open, high, low and closing price. Although this may not seem significant, even when the underlying remains unchanged, choosing one of these options over another can significantly alter the final computed option price. As industry standard, all the calculations were done only with the stock's closing price. Historical price data is also used, when necessary, to compute volatility or to analyze price dynamics over time.

2.1.2 Risk-Free Rate

The risk-free interest rate is a nice concept on paper, but it is rather problematic to find in the real world. In this model, we used as a proxy for it a fund of short-term U.S. T-bills, which provides an approximation for a yearly continuously compounded risk-free rate. This will only be accurate if we are trying to price options on American stocks, but it will fail to capture the actual market risk-free in case we move to a different continent, where the currency fluctuations might change the optimal risk-free choice. Therefore, while using the model, we should keep in mind that if we want to price options not denominated in dollars, we should choose a different proxy for the risk-free rate. It is also important to regularly update the chosen proxy to reflect current market conditions.

2.1.3 Data Limitations

While `yfinance` provides broad access to financial data, it is important to acknowledge certain limitations. The data are not real-time and are subject to delays, missing observations, or reporting inconsistencies. In addition, option liquidity varies significantly across strikes and maturities, which can affect the reliability of bid–ask quotes. These limitations are taken into account when interpreting pricing differences between theoretical models and observed market prices.

Overall, the dataset constructed from yfinance enables a coherent and internally consistent comparison between model-implied option prices and observed market prices, while remaining suitable for a reproducible and transparent Python-based implementation.

2.2 Statistical methods employed

This project employs a combination of analytical pricing formulas, numerical methods, and computational data analysis techniques to value options. All computations are implemented in Python using standard scientific computing libraries where necessary such as NumPy, Pandas, and SciPi.

2.2.1 Analytical Option Pricing

European option prices are computed using the Black–Scholes model, which provides closed-form solutions for call and put options under the assumptions of constant volatility, continuous trading, and no arbitrage. The Black–Scholes formulas are implemented directly in Python using standard mathematical functions and the cumulative distribution function of the standard normal distribution. This analytical framework serves as a benchmark against which numerical and market prices are compared.

2.2.2 Numerical Option Pricing via Binomial Trees

To complement the analytical approach, option prices are also computed using a recombining binomial tree model [3]. The binomial model approximates the stochastic evolution of the underlying asset price in discrete time and prices options through backward induction under the risk-neutral measure.

The binomial tree is constructed with varying numbers of time steps to study the convergence properties of the numerical method. As the number of steps increases, the binomial prices are expected to converge to the corresponding Black–Scholes prices for European options. This convergence behavior is explicitly analyzed and visualized as part of the computational results.

2.2.3 Parameter Estimation and Input Construction

The underlying asset price, strike price, time to maturity, risk-free interest rate, and volatility are constructed from market data. Volatility inputs are obtained from historical return data.

Interest rates are converted to continuously compounded rates to ensure consistency with the Black–Scholes framework. All inputs are carefully aligned in terms of units and conventions to ensure comparability across models.

3 Application Overview

The option pricing application works through a `main.py` script that ties together all the individual modules, which we discuss in the next section. The system follows a clear workflow: it collects user input, runs two different pricing models, displays the results, and provides a visualization. Figure:1 shows how the modules interact.

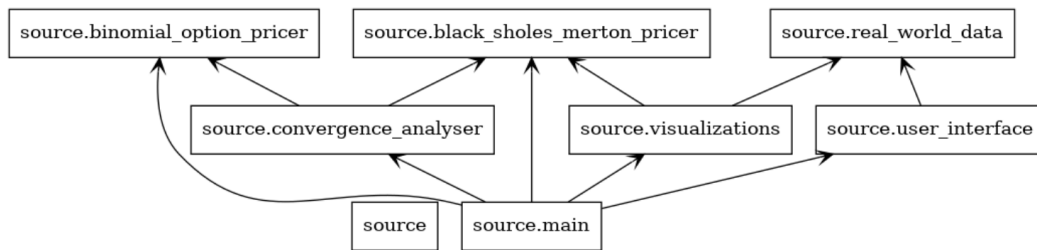


Figure 1: Showing overview of application and how modules interact

3.1 Main Application Program Flow

The application starts when the `main.py` file is run, which serves as the controller of the application. The code starts with user input collection. This is done through the `UserInterface` class prompting for a stock ticker, option type, and moneyness of the option. Then the `RealWorldData` class fetches the required market data. After collecting the data both the binomial tree and Black-Scholes models price the option. The results from the models are then printed to the console and two visualizations are made. Firstly, one that demonstrates the put-call parity and the stock price, and another that shows the convergence of the binomial tree model to the Black-Scholes model.

The following code snippet shows how the `main.py` connects the modules:

```
1 inputs = UserInterface.get_user_inputs()
2
3 pricer_bt = BinomialOptionPricer(
4     S0 = inputs["S0"], K=inputs["K"], T=inputs["T"], R = inputs["R"], n_steps =
5     ↪ inputs["n_steps"], option_type=inputs["option_type"],
6     ↪ sigma=inputs["sigma"]
7 )
8 pricer_bs = BlackScholes(
9     S0 = inputs["S0"], K=inputs["K"], T=inputs["T"], R = inputs["R"],
10    ↪ option_type=inputs["option_type"], vol=inputs["sigma"]
11 )
12
13 price_bs = pricer_bs.compute_blackscholes()
14 option_tree = pricer_bt.price_european()
15
16 print(f"The final option price by the Binomial Model is:
17 ↪ {option_tree[0][0]:.4f}")
18 print(f"The final option price by the Black Scholes Model is: {price_bs:.4f}")
19
20 ConvergenceAnalyser.check_convergence(S0 = inputs["S0"], K=inputs["K"],
21 ↪ T=inputs["T"], R = inputs["R"], sigma=inputs["sigma"],
22 ↪ option_type=inputs["option_type"], max_steps=200, step_increment=5)
23
24 vis = Visual(S0 = inputs["S0"], K=inputs["K"], T=inputs["T"], R = inputs["R"],
25 ↪ vol=inputs["sigma"], ticker_input=inputs["ticker"])
26 vis.plt_put_call_prices()
```

Listing 1: Code snippet showing the flow of the program.

3.2 Key Design Choices

Rather than writing everything in one file, this project is split into a variety of modules and follows several important principles of software design. We follow the separation of concerns principle by making each class responsible for a single, well defined task. For instance, the `UserInterface` handles only user interaction, while the `BlackScholes` class is dedicated solely to pricing the option. This makes the codebase more organized and easier to debug as changes to one area are unlikely to unintentionally affect another.

To ensure we re-use code as much as possible, common functions are centralized rather than duplicated. Both pricing models draw from the same data module, and the `ConvergenceAnalyser` class uses the existing pricing classes rather than implementing them again, avoiding redundant code.

The application uses encapsulation to organize code and control access to internal logic. Static methods, decorated with `@staticmethod` are employed in classes like `UserInterface` and `ConvergenceAnalyser`

where no internal state needs to be maintained. Private methods, indicated by a leading `_` such as `_get_ticker()`, are used to hide implementation details that are only relevant within a class. This approach keeps the public interface interacted with in `main.py` clean and reduces unintended coupling between modules.

4 Overview of Modules

This section provides a detailed look at each module in our option pricing application. We will go through the main classes that make up the system, explaining their purpose, design decisions, and what kind of dependencies they may have. Each module serves a specific role, ensuring we respect the separation of concerns and that our code remains modular. The implementation of the modules were made with best OOP practices in mind.

4.1 `black_scholes_merton_pricer`

The `BlackScholes` class is a direct implementation of the Black-Scholes-Merton model for pricing European options. To use it, we instantiate a `BlackScholes` object with the required parameters: stock price (S_0), strike price (K), time to expiry (T), risk-free rate (R), volatility (vol), and option-type, and then call its pricing method.

As seen in Figure:2, the class has two private methods, `compute_d1()` and `compute_d2()`, which calculate the intermediate d_1 and d_2 values which are used in the final pricing formula. These are only used internally by the class, and as such, following Object Oriented Programming concepts, we decided to make the methods private, not exposing them to other parts of the program.

Another design choice we made was that the intermediate calculations for d_1 and d_2 should be done in their own methods, rather than doing all calculations at once in `compute_blackscholes()`. This choice has three major benefits. Firstly, it makes the code more readable. Secondly, it avoids repetition as we can re-use the computation of d_1 in the computation of d_2 . Lastly, if we needed to adjust or debug the calculation for d_1 or d_2 , we can look at a targeted place, specifically their methods.

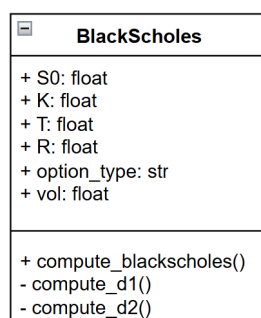


Figure 2: UML class diagram of the `BlackScholes` class

4.1.1 Dependencies & External Libraries

The `BlackScholes` class does not have any dependencies. However, it does make use of external libraries. The libraries it makes use of are `numpy` and `scipy`.

The `numpy` library was used to perform the necessary mathematical calculations done throughout the class. For example, in the `compute_d1()` method `np.log()` and `np.sqrt()` were used. In the main pricing method, `compute_blackscholes()`, `np.exp()` is used to compute the discount factor.

`scipy.stats` provides `norm.cdf()`, the cumulative distribution function for the normal distribution. This function calculates $N(d_1)$ and $N(d_2)$ in the Black-Scholes formula, and is called in the `compute_blackscholes()` method.

Below is a short code snippet showing the usage of both libraries in the `compute_blackscholes()` method:

```

1 def compute_blackscholes(self):
2     if self.option_type=="call":
3         return self.S0 * stats.norm.cdf(self._compute_d1()) - self.K *
           ↳ np.exp(-self.R * self.T) * stats.norm.cdf(self._compute_d2())
4     elif self.option_type=="put":
5         return self.K * np.exp(-self.R * self.T) *
           ↳ stats.norm.cdf(-self._compute_d2()) - self.S0 *
           ↳ stats.norm.cdf(-self._compute_d1())

```

Listing 2: Code snippet showing usage of numpy and scipy.stats libraries

Using these libraries meant we could focus on implementing the Black-Scholes pricer itself, rather than building these mathematical functions from scratch. This approach leverages the libraries available in Python and helped streamline the development process.

4.2 binomial_option_pricer

The `BinomialOptionPricer` class implements the binomial tree method for pricing options. To use it, we create an instance with parameters including stock price, time to expiry, risk-free rate, number of time steps, option type, and volatility. The constructor automatically calculates the tree parameters needed for pricing through its `compute_parameters()` method.

As shown in the UML diagram, Figure 3, the class has two methods for pricing: `build_stock_value_tree()` and `price_european()`. These two methods work together to first build the possible stock price paths and then calculate the option value by working backward from expiration.

A key design decision was using a two-dimensional list to represent the binomial tree. This data structure naturally fits the shape of the tree that is built. We initialize the tree with zeros and then fill in the stock prices using up and down factors.

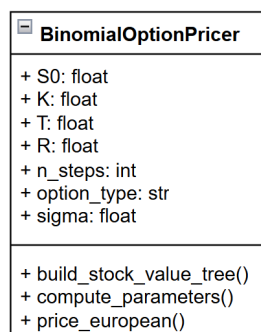


Figure 3: UML class diagram of the `BinomialOptionPricer` class

4.2.1 Implementation Details

The binomial pricing process happens in three main stages. Firstly, the additional parameters are calculated in the `compute_parameters()` method. Here, the up and down factors and the risk-neutral probabilities are calculated based on the stock's historical volatility and how many time steps there are.

After computing the additional parameters, `build_stock_value_tree()` creates the stock price tree by multiplying the initial price by the up and down factors across all possible paths. This is done by traversing the 2D array.

Then, the `price_european()` method computes the option payoff at expiry. With the payoff at expiry, the method works backward through the tree, discounting expected values (using the risk neutral probabilities) at each step, until we reach time 0.

The backward induction step is particularly important. The following code snippet, taken from the `price_european()` method, shows how this works:

```

1 for t in range(self.n_steps - 1, -1, -1):
2     for k in range(t + 1):
3         up_value = option_tree[t + 1][k + 1]
4         down_value = option_tree[t + 1][k]
5         option_tree[t][k] = (self.qu * up_value + self.qd * down_value) / (1 +
            ↪ self.R)

```

Listing 3: Code snippet showing backward induction in the binomial model.

The outer loop (t) goes from back to front, from the second-to-last step to the first step. The inner loop (k) goes through each node at that time step. For each node, we look at the two possible next values (up and down), weight them by the risk-neutral probabilities (self.qu and self.qd), and discount back one time step using $(1 + \text{self.R})$.

4.2.2 Dependencies & External Libraries

The `BinomialOptionPricer` class does not have any external dependencies. However, it does make use of the `numpy` library for mathematical calculations. Specifically, it uses `np.exp()` and `np.sqrt()` to calculate up/down factors and discount rates.

4.3 real_world_data

Having data is the starting point for calculating option prices, without a reliable way to obtain this information, the whole code will fail to deliver its result.

To achieve this, we used the yahoo finance library to retrieve the dataset. This data was imported as a pandas dataframe containing various pieces of information about the stock. However, for our purposes, we have only made use of the stock price on the closing date. We also used NumPy to compute the log returns of the stock in order to calculate the historical volatility. This step is important, as using returns that are not standardised could lead to miscalculating the most important input for our model.

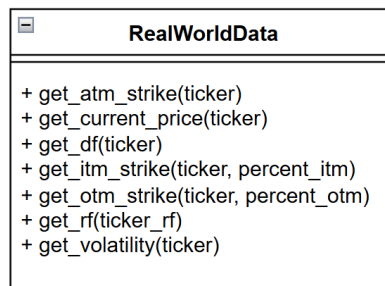


Figure 4: UML class diagram of the RealWorldData class

4.4 visualizations

Having clear and effective visualizations helps us better understand and interpret our results, in this class we have everything to get a good overview of what we have done so far. As shown in the UML diagram, Figure:5, the class takes the stock price, strike price, time till expiry, risk-free rate, RealWorldData class, and volatility as inputs. It's only method is `plt_put_call_prices()` which makes the visualization.

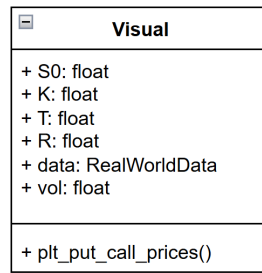


Figure 5: UML class diagram of the Visual class

4.4.1 Dependencies & External Libraries

The import of the BlackScholes class was necessary to compute option values for different underlying stock prices, while the RealWorldData class is used to retrieve the pandas data frame containing market data to be plotted.

Among the main libraries used was `numpy`, required to generate arrays of evenly spaced values needed for pricing calculations. This approach is preferable to Python's built-in `range()` function, as the quantities involved are predominantly floating-point numbers.

Matplotlib represents the core of the class as it is used to produce the final visual output: a graph displaying the BlackScholes prices for both call and put options, along with a second graph showing the historical stock price series. This visualization helps to give a clearer perspective of the relationship between the option prices of different types of options, also known as the put-call parity.

4.5 user_interface

The `UserInterface` class handles all user interaction for the pricing application. Unlike the pricing classes, this class uses only static methods and is never instantiated. This design choice makes sense because we do not need to store the state of an instance and instead return all collected input through the public `get_user_input()` method.

As shown in Figure:6, the class has the one main public method mentioned previously that coordinates the private helper methods. Each helper method handles a specific piece of user input: selecting a stock ticker, choosing an option type, and setting the strike price based on desired moneyness.

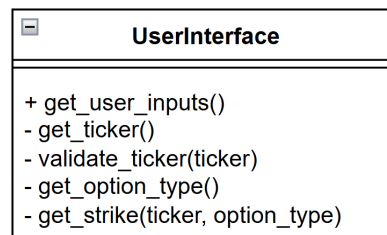


Figure 6: UML class diagram of the UserInterface class

4.5.1 Static & Private Method Design

We made all methods in this class, apart from `get_user_input()`, static for two main reasons. Firstly, the class does not need to store any internal state. We simply collect the user input and return it. Secondly, it allows us to call the methods without needing to instantiate an object first, simplifying the code in our main program.

By using private methods, we respect the separation of concerns. Furthermore, we abstract away unnecessary details in the main. The main just wants the final output and should not be able to see, or have access, to how these final values are calculated. As such, we made the helper methods private.

4.5.2 Input Validation & Real-World Data

A key feature of the application is its integration with real market data through the `RealWorldData` class. Instead of asking users to look up and enter exact stock prices or volatility numbers, the application does this

automatically.

Furthermore, we also implemented user input and data validation, which ensures that the user does not enter a stock ticker that does not exist or inputs a choice that is not listed and would hence cause an error. The following code snippet shows an example of this data validation:

```
1 def _get_option_type():
2     print("\nChoose option type:")
3     print("1. Call option")
4     print("2. Put option\n")
5     print("Enter 1 or 2:\n")
6     while True:
7         choice = input("> ").strip()
8         if choice == "1":
9             return "call"
10        elif choice == "2":
11            return "put"
12        print("Invalid choice! Please enter 1 or 2.")
```

Listing 4: Code snippet showing user input validation for for picking the option type.

The user can choose between option 1 or 2. In case they input something else they are prompted again with an input. This process repeats until the user inputs a correct option. Like this we ensure a correct type of option is passed through the program.

4.5.3 Dependencies & External Libraries

The `UserInterface` class depends on two external components. It depends on the `RealWorldData` class and also on `yfinance`. We use the `RealWorldData` class to fetch all necessary information regarding the input ticker.

The `yfinance` library is used in the `_validate_ticker()` method to quickly check if a stock symbol exists. We fetch just 7 days of data and if we get any data back, the ticker is valid. Otherwise, the ticker does not exist [1].

4.6 convergence analyser

The `ConvergenceAnalyser` class shows that as the number of time steps increases, its price converges to the Black-Scholes price. This class creates a visual comparison between the two pricing methods, showing how they relate to each other.

As shown in the UML diagram for this class, there is just a single static method `check_convergence` that performs the analysis. The method takes the same parameters as both pricing models, plus some parameters that are used in the convergence test, like `max_steps` and `step_increment`.

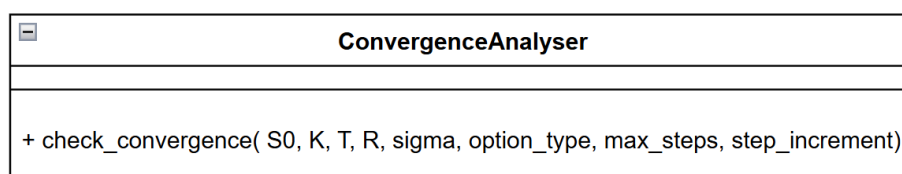


Figure 7: UML class diagram of the `ConvergenceAnalyser` class

4.6.1 Convergence Test

The `check_convergence()` method works as follows. Firstly, it calculates the Black-Scholes price once using the `BlackScholes` class. Then, it calculates binomial prices for an increasing number of time steps

(from 1 to `max_steps`). Finally, it plots all the binomial prices against the constant Black-Scholes price.

This Python code shows the core loop that runs the binomial model at different step counts:

```
1 binomial_prices = []
2 for n in steps:
3     pricer = BinomialOptionPricer(S0, K, T, R, n, option_type, sigma)
4     price = pricer.price_european()[0][0]
5     binomial_prices.append(price)
```

Listing 5: Code snippet showing core loop that computes binomial price at different steps.

The `[0][0]` index accesses the root node of the binomial tree, which contains the final option price. As `n` increases, we expect the binomial price to get closer and closer to the Black-Scholes line on the graph.

4.6.2 Visualization Design

The plot uses two main elements to make the convergence clear. We have a blue line showing the binomial prices at different step counts and a horizontal red line showing the constant Black-Scholes price.

We use `plt.axhline()` to draw the Black-Scholes line, which makes it easy to see how the binomial price approaches it when the step count increases.

4.6.3 Dependencies & External Libraries

The `ConvergenceAnalyser` class depends on all the core modules of our application. The `BlackScholes` class is needed to compute the Black-Scholes price. The `BinomialOptionPricer` class is needed to compute the binomial price. Lastly, the class also needs `matplotlib.pyplot` to be able to make the visualization.

The convergence plot serves as a tool to check the pricers work correctly and also to verify, with our own code, the important concept in derivative pricing that the binomial tree implementation's price converges to the Black-Scholes model's price.

5 Results Overview

The application produces two key visual outputs that both validate the models and illustrate important financial concepts.

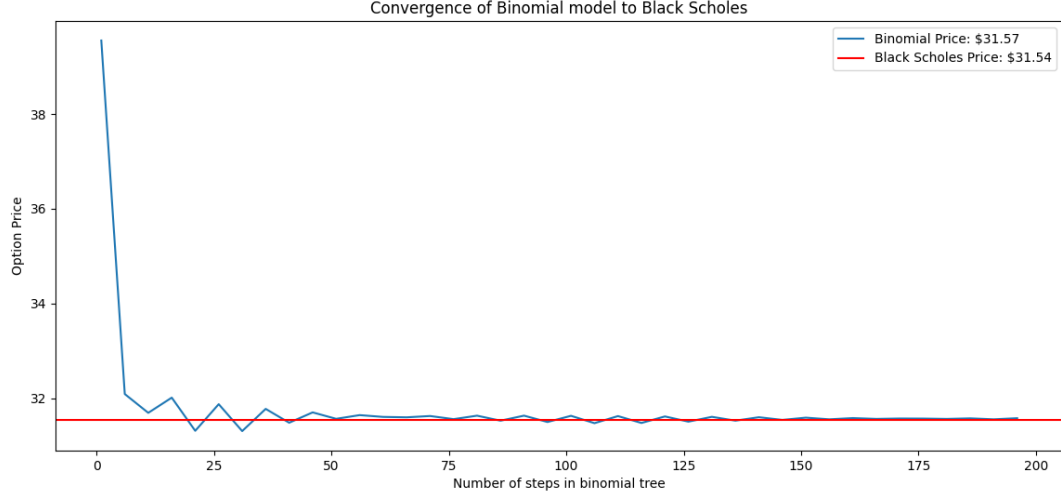


Figure 8: Showing the convergence of the binomial tree model to the Black-Scholes model.

Figure:8 shows that as the number of time steps in the binomial tree increases, the binomial model price converges towards the Black-Scholes price. This validates our implementation and demonstrates a key theoretical the two models. The plots allows users to see how many steps are needed for the binomial model to closely approximate the analytical solution.

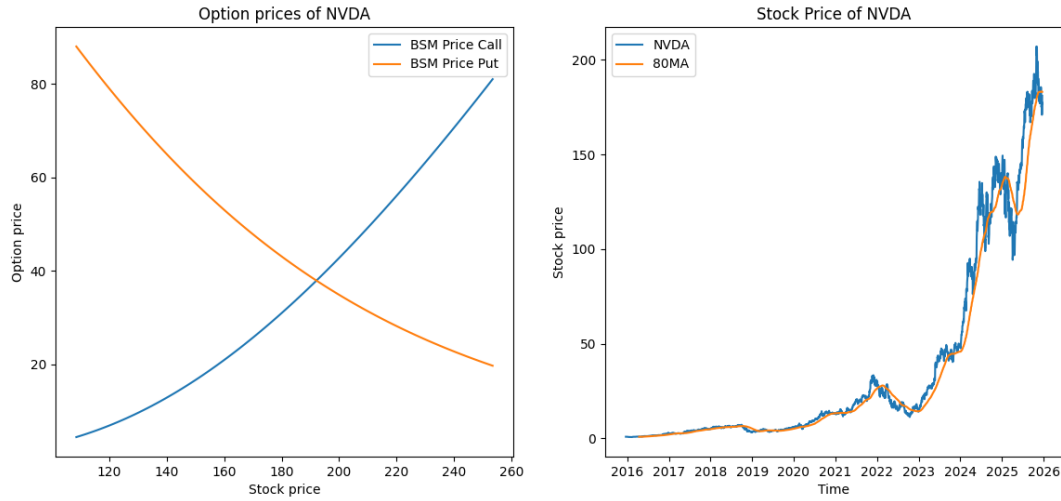


Figure 9: Showing Put-Call parity and the historical stock price and 80 day Moving Average

Figure:9 illustrates put-call parity, which states that the price of a European call option and European put option with the same strike price and expiration should adhere to the following equation: $C - P = S_0 - Ke^{-rT}$. Our visualization shows this relationship holds across.

6 Conclusion

This project developed and implemented a Python-based framework for pricing European options using both analytical and numerical approaches. By integrating the Black-Scholes model with a binomial tree framework, the system enables direct comparison between closed-form solutions and numerical approximations. Its architecture

keeps market data acquisition, pricing, and visualization as distinct components, supporting both transparency and extensibility.

The results confirm the expected convergence properties of the binomial model toward the Black-Scholes price as the number of time steps increases. This behavior provides a useful validation of the numerical implementation and it shows the relationship between continuous time models and their discrete time approximations. The integration of real-world market data further shows the analysis in practical conditions, highlighting both the strengths and the limitations of theoretical pricing models when applied to observed option prices.

Beyond numerical accuracy, the project emphasizes sound software engineering practices within a quantitative finance context. The object-oriented design facilitates reproducibility, testing, and future development, making the framework suitable not only for academic analysis but also as a foundation for more advanced research or applied extensions.

6.1 Limitations

Despite achieving the intended objectives, the code has some limitations itself, which are not given by the code per se, but rather by the way we retrieved the data. As the formulas used to calculate option prices are standardized, obtaining results that are more convergent to what the market is actually pricing requires higher-quality input data.

In this respect, the data imported from Yahoo Finance, despite its high accessibility, remains very superficial in terms of how prices are constructed and calculated. Moreover, since the main input of the model is volatility, a more accurate framework would require the use of volatility forecasting models in order to approximate market expectations as closely as possible. However, not having access to such models, our only way to estimate volatility is through historical time series, which are well known to provide sub-optimal estimates of future volatility.

Despite what has been said, this work should be interpreted as a scholastic approach to interfacing with option pricing models, rather than a tool intended for real-world trading or valuation purposes.

6.2 Future Extensions

Building on the current model, future work may focus on better reflecting the market prices of options through various extensions that address the stated limitations or introduce new features.

A first extension would be the inclusion of American option pricing within the binomial model. Incorporating early exercise would allow direct comparison with observed equity option prices and enable analysis of early exercise premiums, particularly for dividend-paying stocks.

Second, the framework could be extended to compute option sensitivities (Greeks) under both analytical and numerical models. This would enable a better evaluation of model behavior and provide insight into hedging performance and risk management applications.

Third, more advanced volatility modeling could be introduced. Replacing constant volatility assumptions with stochastic volatility models or local volatility surfaces would allow the framework to better capture observed market dynamics.

Fourth, the numerical methods could be expanded beyond binomial trees. Trinomial trees, finite-difference schemes, or Monte Carlo simulations would provide additional perspectives on numerical accuracy, convergence, and computational trade-offs.

In conclusion, while the current implementation is intentionally focused and simplified, it provides a solid foundation for further exploration of option pricing theory, numerical methods, and real-world financial data. The limitations identified highlight important ways for future research, and the modular design ensures that these extensions can be incorporated in a systematic and transparent manner.

References

- [1] alphazwest. How to check if a ticker is yfinance library. *Stack overflow*, 2021.
- [2] Merton Scholes Fisher Black. The pricing of options and corporate liabilities. *Chicago Journals*, 1973.

- [3] Mark Rubinstein John C. Cox, Stephen A. Ross. Option pricing: A simplified approach. *Journal of Financial Economics*, 1979.
- [4] Robert C. Merton. Option pricing when underlying stock returns are discontinuous. *Journal of Financial Economics*, 1976.
- [5] Mark Rubinstein. Implied binomial trees. *The journal of Finance*, 1994.

A LLM Usage Statement

During this project, we used Large Language Models as a learning team rather than relying on them to create the whole project for us.

The LLM usage was done to ask and learn about correct Python Docstrings as well as formatting issues in LaTeX. Prompts were in very similar nature to the, actual ones we asked, listed below:

- I wrote a class in Python what parts of the file should I add Docstrings too?
- How can I put my title between two horizontal bars in Overleaf?
- My image's resolution is very poor in the pdf, how can I make it better?
- How do I add an appendix to LaTeX?
- My import from another file is not being recognized in Python how do I fix this?
- How do I add things to the .gitignore?

B Code Snippets

See the GitHub for all the code and continuous commit history: [Project Github](#).